StreamPulse: Summary

Real-Time Order Delivery Analytics & Data Engineering Platform (Olist Brazilian e-commerce dataset). This is a **historical dataset replay** through a streaming pipeline, not live orders.

1. Data flow

```
Olist CSV (9 files) -> Python producer -> Kafka topic ecommerce_orders (3 partitions)
   -> Spark Structured Streaming (Docker) -> Parquet landing layer + PostgreSQL warehouse
   -> Streamlit dashboard (8 pages)        Frankfurter/ECB API adds BRL->USD to each order
   Airflow runs the finite steps (9 tasks, 2 retries); reconcile.py checks every layer
```

2. What each part does

Part	Role and key fact
Dataset	Olist: 99,441 orders; orders + customers + items + payments joined with a LEFT JOIN (99,441 in, 99,441 out).
Producer	Validates (8 rules), enriches, publishes with acks=all, idempotence, key=order_id. 99,427 valid, 14 rejected.
API	Frankfurter (ECB rates, no key). 3 tries with 1 s / 2 s waits. 100 % of orders enriched; 24,112 used the previous business day.
Kafka	Durable log, 3 partitions, replication 1. order_id key gives an even spread (33.4 / 33.4 / 33.2 %).
Spark	Micro-batches every 5 s (or availableNow). Splits valid/invalid, removes in-batch duplicates, writes Parquet and upserts to PostgreSQL. Checkpoint gives crash recovery. Runs in Docker because Windows lacks winutils.
Parquet	Append-only landing layer: 100,120 rows, 99,427 distinct.
PostgreSQL	Deduplicated warehouse: 99,427 orders, plus reference tables and 11 analytics views.
Airflow	Orchestrates finite tasks only; a stream never ends, so it is not a task. Real run: 9/9 success.
Dashboard	Overview and status lights, dataset, reconciliation, quality, KPIs, API, recent records. Lights are real checks.

2b. Place an order and trace it

The dashboard page **Place a demo order** publishes a new, marked order (`source_category = user_demo`, id `demo-...`) to the same Kafka topic. The unchanged Spark job writes it to Parquet and PostgreSQL, and a trace re-reads each layer for that order id (a stage is ticked only when found). Real run: stored in PostgreSQL in 6.5 s; a resend was a retry duplicate; Airflow's DAG processed orders that waited while Spark was stopped. Historical KPIs are unchanged (99,427 orders).

3. Reliability ideas

- **Dead-letter queue:** rejected records kept with a reason code (14 real orders: 8 delivered without a date, 6 canceled with a delivery date).

- **Duplicates:** record_uid (one source row) and business_hash (one order). Delivery is **at-least-once with an idempotent sink**, never "exactly-once". Re-ingesting 199 messages inserted 0.
- **Recovery:** after a safe stop with 4,995 messages unread, plus new data published while down, restart lost nothing and doubled nothing (18/18 checks).
- **Reconciliation:** expected counts recomputed from the source; exit code 1 on an unexplained difference. Result: all differences zero or explained.

4. Key results (measured)

Kafka messages / Parquet rows / distinct / PostgreSQL	100,514 / 100,120 / 99,427 / 99,427
Revenue BRL / USD estimate / average order	16,006,745 / 4,768,687 / 160.99
Late deliveries / average delivery time	7,826 of 96,470 (8.11 %) / 12.56 days
Tests (all passed)	unit 102, integration 19, Spark 5, Airflow 8, e2e 5

5. Honest limitations

- Replay of 2016-2018 data, not live; USD values are estimates.
- One Kafka broker, one PostgreSQL, Spark in local mode; no monitoring or schema registry.
- Parquet keeps repeats; the warehouse is the deduplicated truth.
- The four live demos and their screenshots are **PENDING_HUMAN**: you must run them.

6. Commands to remember

```
.\scripts\kafka_start.ps1           .\scripts\init_db.ps1
$py -m src.producer.dataset_producer --limit 200
.\scripts\run_stream.ps1 -Trigger availableNow     .\scripts\stop_stream.ps1
$py scripts\reconcile.py            .\scripts\run_dashboard.ps1
.\scripts\airflow_run_dag.ps1 -Dag streampulse_batch -Conf '{"limit": 300}'
```

Full detail: `docs/StreamPulse_Complete_Project_Guide.pdf`.